Analysis of Parallelization Techniques on k-mer Counting for Genomic Sequences

Alex McGinn

October 12, 2026

1 Introduction

In genomic sequence analysis, k-mer counting consists of creating substrings for varying k-lengths from a biological sequence. These sequences consist of the four bases of DNA (A,C,G, and T), which we can count and use to compare DNA samples. They are called k-mers because in chemistry a "mer" is a repeating unit within a larger molecule (in genomics that molecule is mainly DNA). Larger values of k can provide more sequence specific information, while shorter values provide greater sensitivity[2]. For example, if two genes have the same k-mer (e.g. k=15) it would be much more likely they have similar structure in that area. A single base mismatch in one of those larger k-mers would mean many overlapping k-mers would no longer match. Whereas, a shorter k-mer would consider a single character mismatch to have less of an affect on the overall comparison.

Another application of k-mer counting is the estimation of genome size. Marcais and Kingsford explain that k-mer frequency statistics can be used to estimate sequencing coverage and subsequently "derive an estimate of the genome size from c and the total length of the reads" [3].

Although, as k increases, the number of possible k-mers grows exponentially, leading to an increased amount of memory required to store and process k-mer counts. Alternatively, if k is shorter, we may have many matches, which would require the counter variable to have a larger range. For large genomic datasets, these computations become expensive, but there are various methods to accelerate this process using parallelization. The aim of this project was therefore to analyze the effectiveness of CPU/GPU parallelism for these types of tasks.

2 Background

To process a set of k-mer (determined by the k value; e.g. k = 4) we first move a sliding window of size k across the entire sequence and extract every k-mer substring. The first benefit is that we can now compare genomic sequences without simply comparing their complete ordering. Another benefit is that when we break a DNA sequence into many smaller subsequences of a known size, the condensed format allows us to perform computations in parallel, leading to a needed speedup. Although, as the input dataset size and k-value increase, the ability to store the data necessary in main memory becomes increasingly difficult.

To address this problem, one strategy for storing k-mers is to index an array or a similar data structure for each possible sequence. This allows us to predict the amount of main memory necessary to store all possible combinations for a dataset and set k-mer value. For example, a dataset could have a dedicated array that contains all possible k-mer permutation for k-value set equal to five. We know whether this would be possible or not because the number of possible permutations of such a string with 4 possible bases is:

$$4^k$$

e.g. in a 5 k-mer example:

$$4^5 = 1024 \text{ possible subsequences}$$

We could then also have an array that holds the count of each k-mer in the dataset. Therefore, we know that we need 1024 representations of the four possible bases. We can then also predict the size of the array used to count those occurrences. To determine the size necessary, we then need to consider the maximum possible number of occurences a k-mer can have. This will be dictated by frequency, which we cannot outright predict. However, we can be certain of the total number of possible occurrences based on the dataset size. That is because the maximum value will always be equal to how many possible substrings there are, determined by:

$$n - k + 1$$

e.g. in a DNA Sequence of length 66,000 with k-value equal to 5:

$$66000 - 5 + 1 = 65996$$

Which means that we need array elements that go up to at least 65996. In this instance, we would know we now need elements larger than an unsigned short, which would reach capacity at 65,535

2.1 Similar Work

One similar project proposed by Marcais and Kingsford [3] introduced Jellyfish, a parallel k-mer counting algorithm designed to improve performance and memory efficiency when processing large genomic datasets. Jellyfish utilizes a multithreaded, lock-free hash table that allows concurrent updates through atomic operations rather than conventional locking mechanisms. The implementation also leverages compact data representations to reduce memory requirements. Their work demonstrates the importance of synchronization overhead and memory-efficient data structures in parallel k-mer counting, both of which will be considered when evaluating the parallelization strategies proposed in this project.

Deorowicz et al. [1] introduced KMC 2, a parallel k-mer counting algorithm that focuses on reducing memory consumption and disk I/O overhead when processing large genomic datasets. Unlike approaches that rely primarily on concurrent hash-table updates, KMC 2 partitions sequence data into smaller groups using signatures derived from minimizers. These partitions can then be processed using parallel sorting and counting operations, reducing the amount of data that must be held in memory simultaneously. The authors also introduced (k, x)-mers to reduce redundant processing of overlapping k-mers. This approach highlights the importance of memory management, workload distribution, and I/O overhead, which are relevant considerations for the parallelization methodology proposed in this project.

3 Project Objectives

The objective of this project is to analyze the effects of parallelization on k-mer counting for genomic sequences. An initial implementation will be written in C++ performing the k-mer count for a baseline. The algorithm will then be written using OpenMP for parallelization on multiple CPU cores. Another implementation will utilize CUDA for GPU parallelization. MPI may also be evaluated if suitable distributed computing resources are available. There is a possibility of using an old desktop and Amazon EC2 instances to provide this additional resource.

Experiments will vary the dataset size and k-mer size. The performance will then be evaluated in standard methods. Most notably in speedup compared to the serial implementation, memory consumption, and the behavior of the implementation when scaling the dataset and k-mer size.

References

[1] Sebastian Deorowicz, Marek Kokot, Szymon Grabowski, and Agnieszka Debudaj-Grabysz. Kmc 2: fast and resource-frugal k-mer counting. *Bioinformatics*, 31(10):1569–1576, 2015.

[2] John A. Lees, Minna Vehkala, Niko Välimäki, et al. Sequence element enrichment analysis to determine the genetic basis of bacterial phenotypes. *Nature Communications*, 7:12797, 2016.

[3] Guillaume Marcais and Carl Kingsford. A fast, lock-free approach for efficient parallel counting of occurrences of k-mers. *Bioinformatics*, 27(6):764–770, 2011.